Name: Saai Vardhan D
Registration number : 192525205

CSA 0421 - OPERATING SYSTEM

Experiment 16

Develop a C program for implementing random access file for processing the employee details.

Executable C code :

```
#include <stdio.h>

struct employee {
    int id;
    char name[20];
    float salary;
};

int main() {
    FILE *fp;
    struct employee e;
    int n, i, r;
    float newsal;

    fp = fopen("employee.dat", "wb+");
    if (fp == NULL) {
        printf("Cannot open file\n");
        return 1;
    }

    printf("Enter number of employees: ");
    scanf("%d", &n);
    for (i = 1; i <= n; i++) {
        printf("Enter id, name and salary of employee %d: ", i);
        scanf("%d %s %f", &e.id, e.name, &e.salary);
        fwrite(&e, sizeof(e), 1, fp);
    }

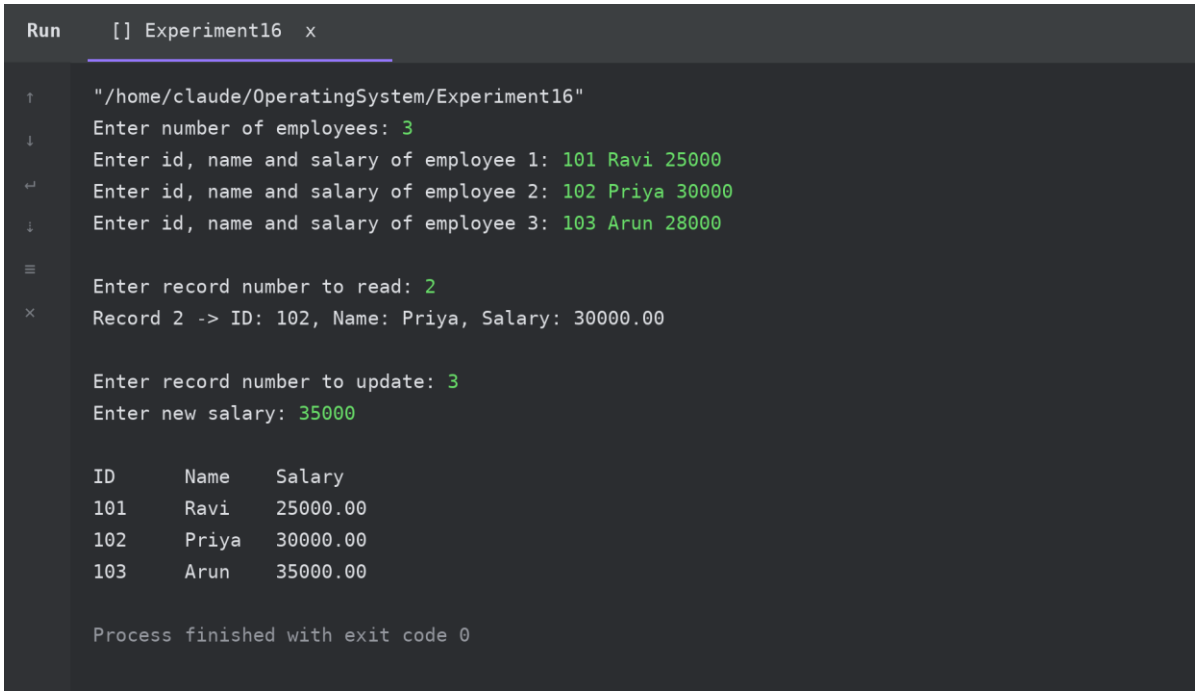
    // read any record directly using its record number
    printf("\nEnter record number to read: ");
    scanf("%d", &r);
    fseek(fp, (r - 1) * sizeof(e), SEEK_SET);
    fread(&e, sizeof(e), 1, fp);
    printf("Record %d -> ID: %d, Name: %s, Salary: %.2f\n", r, e.id, e.name,
e.salary);

    // update any record directly
    printf("\nEnter record number to update: ");
    scanf("%d", &r);
    printf("Enter new salary: ");
    scanf("%f", &newsal);
    fseek(fp, (r - 1) * sizeof(e), SEEK_SET);
    fread(&e, sizeof(e), 1, fp);
    e.salary = newsal;
    fseek(fp, (r - 1) * sizeof(e), SEEK_SET);
    fwrite(&e, sizeof(e), 1, fp);

    printf("\nID\tName\tSalary\n");
    rewind(fp);
    while (fread(&e, sizeof(e), 1, fp) == 1)
        printf("%d\t%s\t%.2f\n", e.id, e.name, e.salary);
```

```
    fclose(fp);
    return 0;
}
```

Output Screenshot:



Experiment 17

Illustrate the deadlock avoidance concept by simulating Banker's algorithm with C.

Executable C code :

```
#include <stdio.h>

int n, m;
int alloc[10][10], max[10][10], need[10][10], avail[10];

// returns 1 and prints the safe sequence if the system is in a safe state
int is_safe(int print) {
    int work[10], finish[10] = {0}, seq[10], count = 0, i, j, found;

    for (j = 0; j < m; j++) work[j] = avail[j];
    while (count < n) {
        found = 0;
        for (i = 0; i < n; i++) {
            if (!finish[i]) {
                for (j = 0; j < m && need[i][j] <= work[j]; j++);
                if (j == m) {                       // process i can finish
                    for (j = 0; j < m; j++) work[j] += alloc[i][j];
                    finish[i] = 1;
                    seq[count++] = i;
                    found = 1;
                }
            }
        }
        if (!found) return 0;
    }
    if (print) {
        printf("System is in a SAFE state.\nSafe sequence: ");
        for (i = 0; i < n; i++) printf("P%d%s", seq[i], i < n - 1 ? " -> " :
"\n");
    }
    return 1;
}

int main() {
    int i, j, p, req[10], ok;

    printf("Enter number of processes and resource types: ");
    scanf("%d %d", &n, &m);
    printf("Enter Allocation matrix:\n");
    for (i = 0; i < n; i++) for (j = 0; j < m; j++) scanf("%d", &alloc[i][j]);
    printf("Enter Max matrix:\n");
    for (i = 0; i < n; i++) for (j = 0; j < m; j++) scanf("%d", &max[i][j]);
    printf("Enter Available resources: ");
    for (j = 0; j < m; j++) scanf("%d", &avail[j]);

    for (i = 0; i < n; i++)
        for (j = 0; j < m; j++) need[i][j] = max[i][j] - alloc[i][j];

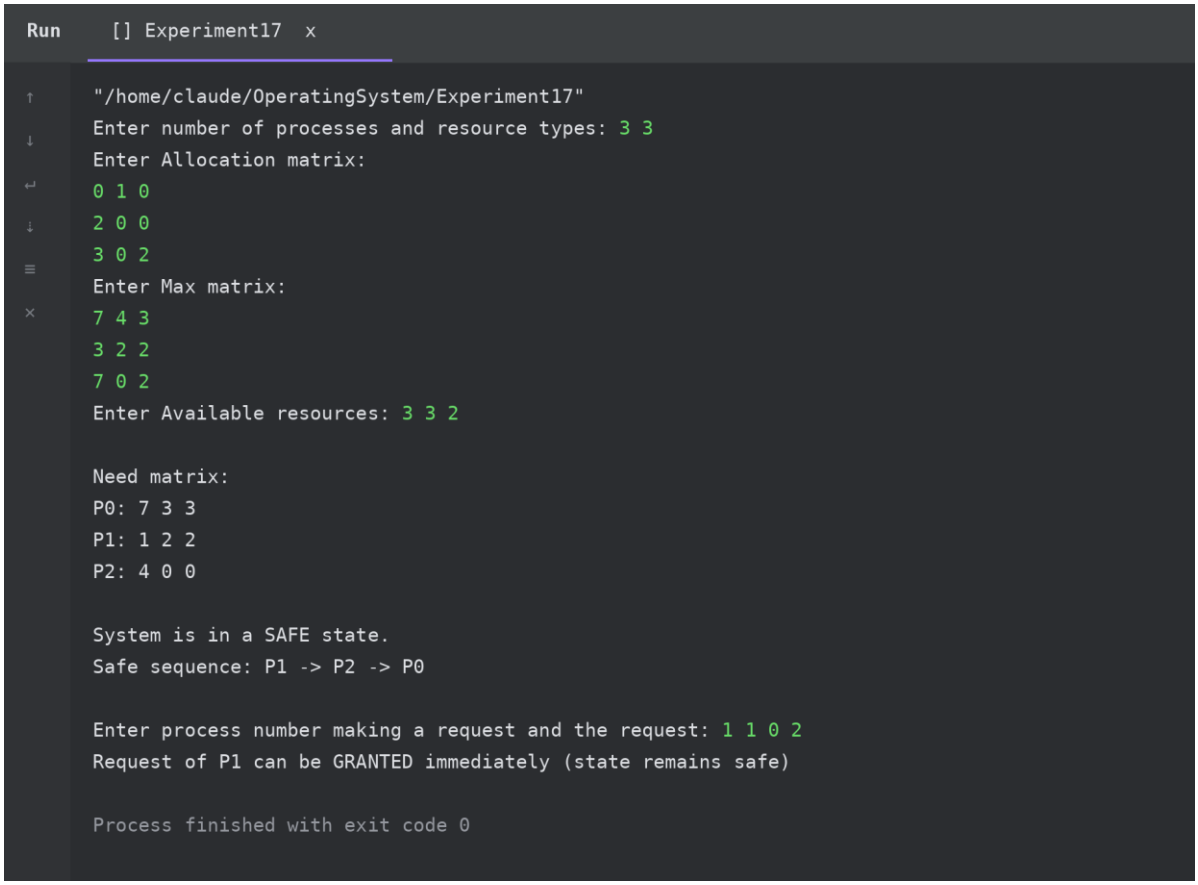
    printf("\nNeed matrix:\n");
    for (i = 0; i < n; i++) {
        printf("P%d: ", i);
        for (j = 0; j < m; j++) printf("%d ", need[i][j]);
        printf("\n");
    }
    printf("\n");
    if (!is_safe(1)) {
        printf("System is NOT in a safe state (deadlock possible)\n");
        return 0;
    }

    printf("\nEnter process number making a request and the request: ");
    scanf("%d", &p);
```

```
    for (j = 0; j < m; j++) scanf("%d", &req[j]);

    ok = 1;
    for (j = 0; j < m; j++)
        if (req[j] > need[p][j] || req[j] > avail[j]) ok = 0;
    if (!ok) {
        printf("Request cannot be granted now (exceeds need or available)\n");
        return 0;
    }
    // pretend to grant the request and test whether the state stays safe
    for (j = 0; j < m; j++) {
        avail[j] -= req[j];
        alloc[p][j] += req[j];
        need[p][j] -= req[j];
    }
    if (is_safe(0))
        printf("Request of P%d can be GRANTED immediately (state remains
safe)\n", p);
    else
        printf("Request of P%d must WAIT (granting it would be unsafe)\n", p);
    return 0;
}
```

Output Screenshot:



Experiment 18

Construct a C program to simulate producer-consumer problem using semaphores.

Executable C code :

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define SIZE 3

int buffer[SIZE], in = 0, out = 0, N;
sem_t empty, full;
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

void *producer(void *arg) {
    int i;
    for (i = 1; i <= N; i++) {
        sem_wait(&empty);                  // wait for an empty slot
        pthread_mutex_lock(&mutex);
        buffer[in] = i;
        printf("Producer produced item %d at position %d\n", i, in);
        in = (in + 1) % SIZE;
        pthread_mutex_unlock(&mutex);
        sem_post(&full);                   // one more full slot
        usleep(100000);
    }
    return NULL;
}

void *consumer(void *arg) {
    int i, item;
    for (i = 1; i <= N; i++) {
        sem_wait(&full);                   // wait for a full slot
        pthread_mutex_lock(&mutex);
        item = buffer[out];
        printf("Consumer consumed item %d from position %d\n", item, out);
        out = (out + 1) % SIZE;
        pthread_mutex_unlock(&mutex);
        sem_post(&empty);                  // one more empty slot
        usleep(300000);
    }
    return NULL;
}

int main() {
    pthread_t p, c;

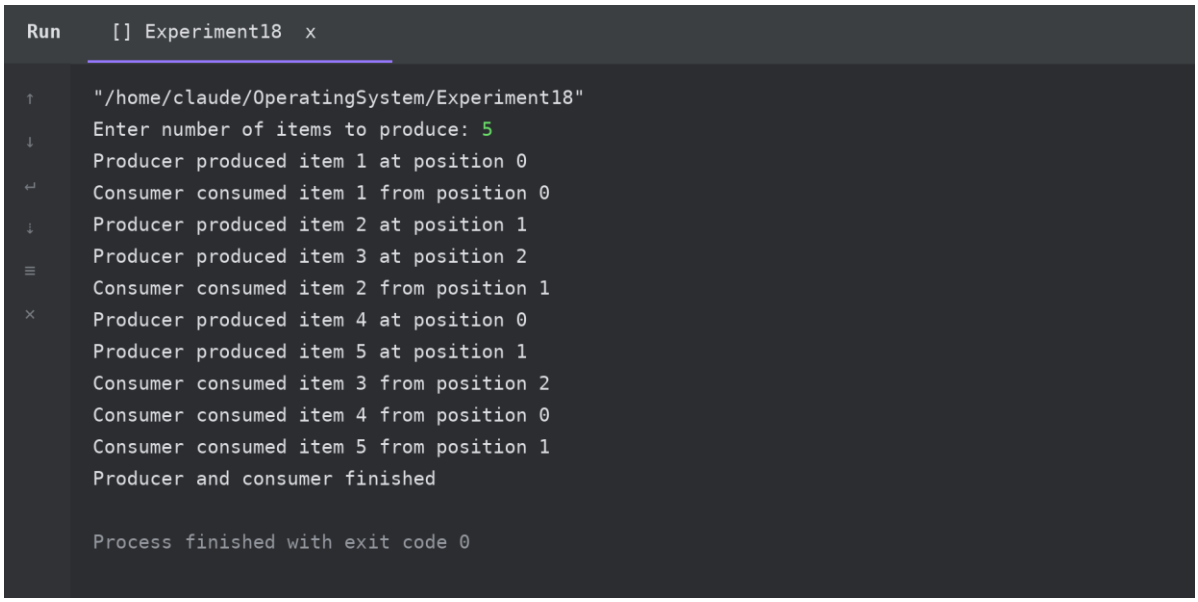
    printf("Enter number of items to produce: ");
    scanf("%d", &N);

    sem_init(&empty, 0, SIZE);
    sem_init(&full, 0, 0);

    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);
    pthread_join(p, NULL);
    pthread_join(c, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);
    printf("Producer and consumer finished\n");
    return 0;
}
```

Output Screenshot:



Experiment 19

Design a C program to implement process synchronization using mutex locks.

Executable C code :

```c
#include <stdio.h>
#include <pthread.h>

int counter = 0, n;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

void *increment(void *arg) {
    int i;
    for (i = 0; i < n; i++) {
        pthread_mutex_lock(&lock);      // enter critical section
        counter++;
        pthread_mutex_unlock(&lock);    // leave critical section
    }
    return NULL;
}

int main() {
    pthread_t t[3];
    int i;

    printf("Enter number of increments per thread: ");
    scanf("%d", &n);

    for (i = 0; i < 3; i++) pthread_create(&t[i], NULL, increment, NULL);
    for (i = 0; i < 3; i++) pthread_join(t[i], NULL);

    printf("Expected counter value = %d\n", 3 * n);
    printf("Actual counter value   = %d\n", counter);
    if (counter == 3 * n)
        printf("Synchronization successful: no race condition\n");
    return 0;
}
```

Output Screenshot:



Experiment 20

Construct a C program to simulate Reader-Writer problem using Semaphores.

Executable C code :

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

sem_t wrt;
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
int readcount = 0, data = 0;

void *reader(void *arg) {
    int id = *(int *)arg;

    pthread_mutex_lock(&mutex);
    readcount++;
    if (readcount == 1) sem_wait(&wrt);     // first reader blocks writers
    pthread_mutex_unlock(&mutex);

    printf("Reader %d is reading, data = %d\n", id, data);
    usleep(300000);
    printf("Reader %d finished reading\n", id);

    pthread_mutex_lock(&mutex);
    readcount--;
    if (readcount == 0) sem_post(&wrt);     // last reader lets writers in
    pthread_mutex_unlock(&mutex);
    return NULL;
}

void *writer(void *arg) {
    int id = *(int *)arg;

    sem_wait(&wrt);                          // exclusive access
    data++;
    printf("Writer %d is writing, data = %d\n", id, data);
    usleep(300000);
    printf("Writer %d finished writing\n", id);
    sem_post(&wrt);
    return NULL;
}

int main() {
    pthread_t rt[5], wt[5];
    int r, w, i, id[5] = {1, 2, 3, 4, 5};

    printf("Enter number of readers and writers: ");
    scanf("%d %d", &r, &w);
    sem_init(&wrt, 0, 1);

    // start readers and writers one after another, 0.1 second apart
    for (i = 0; i < r || i < w; i++) {
        if (i < r) { pthread_create(&rt[i], NULL, reader, &id[i]);
usleep(100000); }
        if (i < w) { pthread_create(&wt[i], NULL, writer, &id[i]);
usleep(100000); }
    }
    for (i = 0; i < r; i++) pthread_join(rt[i], NULL);
    for (i = 0; i < w; i++) pthread_join(wt[i], NULL);

    sem_destroy(&wrt);
    return 0;
}
```

Output Screenshot:

